

The SOG Bible

Current working-tree edition

Last updated: 2026-06-22

Editable source: docs/SOG_BIBLE.md

Generated PDF: The_SOG_Bible_v2.pdf

What This Document Is

This is the current canonical end-to-end reference for the Synthetic Occupancy Generator repository as it exists today. It replaces older SOG Bible PDFs and older guide sections that assumed root-level scripts/, src/sog_phase2/, or Data/phase2_params/ paths.

Use this file when you need the current workflow, current paths, current Phase 1 configuration behavior, current Phase 2 scenario contract, and current output artifacts in one place.

The current repository is phase-scoped:

- Phase 1 code and scripts live under phase1/.
- Phase 2 code, scripts, scenarios, tests, docs, priors, and run artifacts live under phase2/.
- The Streamlit frontend and agent helpers live under frontend/.

Source Of Truth Files

When this Bible and another document disagree, use the code and these active references in this order:

1. Phase 1 config validation: phase1/src/sog_phase1/config.py
2. Phase 1 generator behavior: phase1/src/sog_phase1/generator.py
3. Phase 1 YAML: phase1/configs/phase1.yaml
4. Phase 1 parameter notes: phase1/configs/phase1_config_parameters.md
5. Phase 2 scenario YAMLs: phase2/scenarios/*.yaml
6. Phase 2 scenario template: phase2/scenarios/_working_scenario_template.yaml
7. Phase 2 scenario catalog: phase2/scenarios/catalog.yaml
8. Phase 2 engine: phase2/src/sog_phase2/
9. Phase 2 output contract: phase2/src/sog_phase2/output_contract.py
10. Phase 2 run-folder reference: phase2/runs/README.md

Current Repository Map

```
SOG/  
|-- phase1/  
|   |-- Addresses/  
|   |-- Names/  
|   |-- Data/  
|   |-- configs/phase1.yaml  
|   |-- configs/phase1_config_parameters.md  
|   |-- scripts/
```

```
| |-- src/sog_phase1/
| |-- tests/
| |-- prepared/           # generated, gitignored
| |-- outputs/           # generated, gitignored
| '-- outputs_phase1/     # canonical Phase 2 baseline location
|-- phase2/
| |-- Data/phase2_params/
| |-- docs/
| |-- scenarios/
| |-- scripts/
| |-- src/sog_phase2/
| |-- tests/
| '-- runs/               # generated run folders
|-- frontend/
|-- tests/               # frontend tests
|-- docs/
|-- requirements.txt
|-- run_frontend.ps1
'-- The_SOG_Bible_v2.pdf
```

What SOG Does

SOG is a deterministic synthetic data platform for entity-resolution and record-linkage benchmarking.

It creates a known synthetic world, changes that world through simulated life and household events, emits one or more noisy operational views of the world, and ships the answer keys needed to evaluate an ER system.

The pipeline has two primary phases:

1. Phase 1 generates the baseline person-and-address population.
2. Phase 2 selects scenario participants, simulates truth events, emits observed datasets, and validates the run package.

The key design principle is separation:

- Phase 1 creates stable people and baseline administrative rows.
- Phase 2 creates scenario-specific temporal truth and observed data.
- Output contracts and metadata make each run auditable and reproducible.

Install And Setup

Run commands from the repository root.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

requirements.txt is the dependency source of truth in this checkout. It already contains runtime packages, the frontend packages, and pytest.

Phase 1 In Plain Terms

Phase 1 answers these questions:

- How many people exist?

- What stable identity and demographic attributes do they have?
- How many flat-file rows represent each person?
- How much legitimate name collision, nickname variation, mailing-address divergence, and row redundancy should be present?

The main config is:

phase1/configs/phase1.yaml

The current default shape is:

```
phase1:
  n_people: 10000
  n_records: 14000
  seed: 20260303
  output:
    format: csv
    path: outputs/Phase1_people_addresses.csv
    chunk_size: 5000
```

Current Phase 1 Defaults

Setting	Current value	Meaning
'phase1.n_people'	'10000'	Unique people, therefore unique 'PersonKey' values.
'phase1.n_records'	'14000'	Requested base output rows before nickname backup rows.
'phase1.seed'	'20260303'	Reproducibility seed.
'phase1.output.format'	'csv'	Main file format.
'phase1.output.path'	'outputs/Phase1_people_addresses.csv'	Path relative to 'phase1/'.
'phase1.output.chunk_size'	'5000'	Write batching size.
'phase1.redundancy.enabled'	'true'	Allows repeated rows for the same person.
'phase1.redundancy.shape'	'heavy_tail'	Concentrates extra records among fewer people.
'phase1.nicknames.enabled'	'true'	Emits display first names that can differ from formal first names.
'phase1.nicknames.mode'	'per_record'	Nickname display can vary row by row.
'phase1.nicknames.usage_pct'	'35.0'	Target nickname usage rate.
'phase1.name_duplication.full_name_people_pct'	'58.0'	Target people participating in exact formal full-name collisions.

Phase 1 Output Formats

phase1.output.format accepts:

- csv
- parquet
- txt
- xlsx
- excel as an alias for xlsx

Compatibility typo aliases exist in the code for older configs, but new configs should use the canonical values above.

Format behavior:

- csv writes Phase1_people_addresses.csv.
- txt writes a tab-delimited Phase1_people_addresses.txt.
- xlsx and excel write Phase1_people_addresses.xlsx with a Phase1 worksheet.
- parquet writes chunk files under a <output_stem>_parts/ directory.

Important Phase 1 Behaviors

n_people and n_records are not the same thing. n_people controls entity count. n_records controls flat-file row count. With the current default, Phase 1 generates 10,000 people and requests 14,000 base rows, so some people appear more than once.

When redundancy is enabled:

$$n_people * min_records_per_entity \leq n_records \leq n_people * max_records_per_entity$$

When redundancy is disabled, n_records must equal n_people.

Nickname behavior can add formal backup rows. If a person receives a nickname row but does not already have a formal-name row, the generator appends a formal backup row. Check records_requested, records_written, and formal_copy_records_added in the manifest and quality report.

Name duplication is entity-level collision pressure. It does not duplicate rows. It makes different people legitimately share first names, last names, or exact formal full names.

Residence dates in Phase 1 are sampled per row. Coherent household movement and event-driven residence intervals are Phase 2 responsibilities.

Build The Phase 1 Baseline

Run:

```
python phase1/scripts/build_prepared.py
python phase1/scripts/generate_phase1.py --overwrite
```

Primary generated artifacts:

```
phase1/outputs/Phase1_people_addresses.csv
phase1/outputs/Phase1_people_addresses.manifest.json
phase1/outputs/Phase1_people_addresses.quality_report.json
```

Phase 2 uses the canonical baseline under phase1/outputs_phase1/, so copy the fresh Phase 1 artifacts there when you want Phase 2 to consume them:

```
Copy-Item phase1/outputs/Phase1_people_addresses.csv phase1/outputs_phase1/Phase1_people_addresses.csv -Force
Copy-Item phase1/outputs/Phase1_people_addresses.manifest.json phase1/outputs_phase1/Phase1_people_addresses.manifest.json -Force
Copy-Item phase1/outputs/Phase1_people_addresses.quality_report.json phase1/outputs_phase1/Phase1_people_addresses.quality_report.json -Force
```

Phase 2 In Plain Terms

Phase 2 answers these questions:

- Which people from Phase 1 enter this scenario?
- What events happen to them over time?
- Which systems observe them, at which snapshot dates, and with what coverage?
- Which fields are corrupted, masked, missing, duplicated, or drifted?
- What answer keys prove which observed records refer to the same real person?

Phase 2 is now self-contained under:

phase2/

The current implementation lives under:

phase2/src/sog_phase2/

The current CLI entrypoints live under:

phase2/scripts/

The current parameter package lives under:

phase2/Data/phase2_params/

Build Phase 2 Priors

Run:

```
python phase2/scripts/build_phase2_params.py
```

The priors package contains mobility, household, marriage/divorce, fertility, source, and provenance files used by Phase 2.

Run A Phase 2 Scenario

The preferred one-command path is:

```
python phase2/scripts/run_phase2_pipeline.py --scenario single_movers --overwrite
```

Use --rebuild-population when selection or filtering changed and you want to discard the cached scenario_population.parquet:

```
python phase2/scripts/run_phase2_pipeline.py --scenario single_movers --overwrite --rebuild-population
```

The staged equivalent is:

```
python phase2/scripts/generate_phase2_truth.py --run-id 2026-03-10_single_movers_seed20260310 --overwrite
python phase2/scripts/generate_phase2_observed.py --run-id 2026-03-10_single_movers_seed20260310 --overwrite
python phase2/scripts/validate_phase2_outputs.py --run-id 2026-03-10_single_movers_seed20260310
```

Run IDs use:

YYYY-MM-DD_<scenario_id>_seed<seed>

Example:

2026-03-10_single_movers_seed20260310

Important Cache Rule

--overwrite replaces output artifacts, but by itself it does not rebuild an existing scenario_population.parquet.

If you edit selection filters, sample size, age bins, genders, residence types, or similar participant-selection logic in a scenario YAML, use:

```
--rebuild-population
```

The selection audit lives in:

```
phase2/runs/<run_id>/scenario_selection_log.json
```

Use that file to confirm selected counts, filter survivor counts, selection seed, and selected PersonKey checksum.

Scenario YAML Contract

Canonical scenario YAMLs live under:

```
phase2/scenarios/
```

The editable starter template is:

```
phase2/scenarios/_working_scenario_template.yaml
```

Copy that file to phase2/scenarios/<scenario_id>.yaml, remove the _working_ prefix, and set scenario_id to match the filename.

Top-level scenario sections:

```
scenario_id: single_movers
seed: 20260310
phase1: { data_path: ..., manifest_path: ... }
parameters: { ... }
selection: { ... }
constraints: { ... }
simulation: { ... }
emission: { ... }
quality: { ... }
```

Only scenario_id, seed, and phase1 are strictly required. Other sections fall back to defaults, but shipped scenarios define the important knobs explicitly.

Selection

Selection is entity-centric. It collapses Phase 1 rows to people, attaches deterministic latent propensity scores, filters candidates, samples the cohort, and writes scenario_population.parquet.

Common keys:

- selection.sample.mode: all, count, or pct

- selection.sample.value: count or percentage
- selection.filters.age_bins
- selection.filters.genders
- selection.filters.ethnicities
- selection.filters.residence_types
- selection.filters.redundancy_profiles
- selection.filters.mobility_propensity_buckets
- selection.thresholds.mobility_low_max
- selection.thresholds.mobility_high_min
- selection.thresholds.trait_low_max
- selection.thresholds.trait_high_min

Parameters

Scenario parameters are annual event rates:

- move_rate_pct
- cohabit_rate_pct
- birth_rate_pct
- divorce_rate_pct
- leave_home_rate_pct
- split_rate_pct as a backward-compatible alias for leave-home behavior
- use_priors_for_unspecified_rates

The simulator converts annual rates to per-step probabilities based on simulation.granularity.

Constraints

Constraints gate and validate realism:

- min_marriage_age
- max_partner_age_gap
- partner_age_gap_distribution
- fertility_age_range.min
- fertility_age_range.max
- allow_underage_marriage
- allow_child_lives_alone
- enforce_non_overlapping_residence_intervals

Simulation

The current simulation keys are:

- granularity: monthly or daily
- start_date: YYYY-MM-DD
- periods: positive integer

Monthly is the recommended default. Daily runs create many more event opportunities and can be slower.

Emission

Emission controls observed source files. The canonical schema supports one, two, or more datasets:

```

emission:
  crossfile_match_mode: one_to_one
  overlap_entity_pct: 70.0
  datasets:
    - dataset_id: A
      filename: DatasetA.csv
      snapshot: simulation_start
      appearance_pct: 85.0
      duplication_pct: 4.0
      record_count: null
      noise: { ... }
    - dataset_id: B
      filename: DatasetB.csv
      snapshot: simulation_end
      appearance_pct: 90.0
      duplication_pct: 6.0
      record_count: null
      noise: { ... }

```

Accepted crossfile_match_mode values:

- single_dataset
- one_to_one
- one_to_many
- many_to_one
- many_to_many

Legacy two-source A/B keys are still accepted for compatibility, but new scenarios should use emission.datasets[*].

Noise Dials

Noise is configured per dataset. Current dials are percentages applied per record:

- name_typo_pct
- dob_shift_pct
- ssn_mask_pct
- phone_mask_pct
- address_missing_pct
- middle_name_missing_pct
- phonetic_error_pct
- ocr_error_pct
- date_swap_pct
- zip_digit_error_pct
- nickname_pct
- suffix_missing_pct

Built-In Scenario Catalog

The machine-readable catalog is phase2/scenarios/catalog.yaml.

```

| Scenario | Topology | Cardinality | Truth event surface | Current status |
| --- | --- | --- | --- | --- |
| 'single_movers' | pairwise | 'one_to_one' | 'MOVE' | supported |
| 'clean_baseline_linkage' | pairwise | 'one_to_one' | 'MOVE' | supported |
| 'couple_merge' | pairwise | 'one_to_many' | 'COHABIT', 'MOVE' | supported |
| 'family_birth' | pairwise | 'many_to_one' | 'BIRTH', 'COHABIT' | supported |
| 'divorce_custody' | pairwise | 'many_to_many' | 'DIVORCE', 'COHABIT' | supported |
| 'roommates_split' | pairwise | 'one_to_many' | 'LEAVE_HOME', 'MOVE', 'COHABIT' | supported |

```

```
| 'high_noise_identity_drift' | pairwise | 'one_to_one' | 'MOVE' | supported |
| 'low_overlap_sparse_coverage' | pairwise | 'one_to_one' | 'MOVE' | supported |
| 'asymmetric_source_coverage' | pairwise | 'one_to_one' | 'MOVE' | supported |
| 'high_duplication_dedup' | single dataset | 'dedup' | 'MOVE' background | supported |
| 'three_source_partial_overlap' | N-way | 'one_to_one' | 'MOVE' | supported |
| 'name_change_lifecycle' | pairwise | 'one_to_one' | 'NAME_CHANGE' | supported |
| 'death_survivor_persistence' | pairwise | 'one_to_one' | 'DEATH' | supported |
| 'adoption_blended_family' | pairwise | 'one_to_one' | 'ADOPTION' | supported |
```

Parameterized but supported families:

- custom_single_dataset_dedup
- custom_pairwise_linkage
- custom_multi_dataset_linkage
- custom lifecycle variants through parameters.name_change_rate_pct, parameters.death_rate_pct, and parameters.adoption_rate_pct

These events are active in the truth event grammar and simulator. NAME_CHANGE and ADOPTION replay updated names into later observed snapshots and RecordType=residence_timeline rows. DEATH closes active household/residence intervals and records IsDeceased / DeathDate in the truth person table.

Phase 2 Run Folder Anatomy

Each run lands under:

phase2/runs/<run_id>/

Truth-layer Parquet files:

- truth_people.parquet
- truth_households.parquet
- truth_household_memberships.parquet
- truth_residence_history.parquet
- truth_events.parquet
- scenario_population.parquet

Observed-layer CSV files:

- one or more scenario-defined dataset CSVs, such as DatasetA.csv, DatasetB.csv, or custom filenames
- entity_record_map.csv
- masterDataset.csv
- truth_crosswalk.csv for two-dataset pairwise runs
- truth_crosswalk<left><right>.csv files for multi-dataset pairwise views

Metadata files:

- scenario.yaml
- scenario_selection_log.json
- manifest.json
- quality_report.json
- README.md

Answer Keys

Use entity_record_map.csv as the canonical answer key.

It maps every emitted observed record to the real person:

PersonKey, DatasetId, RecordKey

Two observed records refer to the same real-world entity if they share the same PersonKey. For clustering evaluation, group observed records by PersonKey.

truth_crosswalk.csv is a backward-compatible A/B pairwise view for two-dataset runs. It is useful for pairwise comparisons, but it is positional and can show blank opposite-side cells when within-file duplicates exist. For transitive cluster truth, prefer entity_record_map.csv.

For three-or-more dataset runs, pairwise crosswalk files can be emitted per dataset pair, but entity_record_map.csv remains the canonical truth bridge.

masterDataset.csv

Every current Phase 2 run emits masterDataset.csv.

It has two row types:

- RecordType=source_snapshot: stacks the observed source dataset rows and adds PersonKey.
- RecordType=residence_timeline: adds DatasetId=TIMELINE rows from truth_residence_history.parquet.

The source snapshot section is the best convenience table when you want all observed inputs in one CSV. The timeline section is useful when you need to inspect intermediate simulated address intervals that do not necessarily appear as source snapshots.

Observed source rows are ordered through the entity_record_map.csv bridge and stable PersonKey-aware sorting so combined outputs do not depend on arbitrary file order.

Quality And Validation

Phase 1 writes:

- Phase1_people_addresses.manifest.json
- Phase1_people_addresses.quality_report.json

Phase 2 writes:

- manifest.json
- quality_report.json
- validation output from phase2/scripts/validate_phase2_outputs.py

Phase 2 quality checks include:

- required file presence
- schema checks
- run ID, scenario ID, seed, and Phase 1 input consistency
- selected-entity counts against scenario_population.parquet
- event grammar checks
- constraint validation

- household-size threshold checks
- observed artifact discovery

Frontend

The active frontend entrypoint is:

```
frontend/chatbot_production.py
```

Launch through:

```
.\run_frontend.ps1
```

Manual command:

```
python -u -m streamlit run frontend/chatbot_production.py --server.headless true
```

Frontend helpers should prefer the current package path:

```
frontend.visualizations.core
```

Test Commands

Full suite:

```
python -m pytest -q
```

Phase 1 focused tests:

```
python -m pytest -q phase1/tests
```

Phase 2 focused tests:

```
python -m pytest -q phase2/tests
```

Frontend focused tests:

```
python -m pytest -q tests
```

Scenario regression:

```
python -m pytest -q phase2/tests/test_phase2_scenario_regression.py
```

If Windows numerical libraries fail with a thread-start error, retry with:

```
$env:OMP_NUM_THREADS='1'  
$env:OPENBLAS_NUM_THREADS='1'  
$env:MKL_NUM_THREADS='1'  
$env:NUMEXPR_NUM_THREADS='1'  
python -m pytest -q
```

Common Mistakes

Do not run old root-level commands such as:

```
python scripts/generate_phase1.py
```

Use:

```
python phase1/scripts/generate_phase1.py
```

Do not assume --overwrite rebuilds selected people. Use --rebuild-population after participant-selection edits.

Do not evaluate clusters from truth_crosswalk.csv alone when duplicates exist. Use entity_record_map.csv.

Do not commit generated outputs such as phase1/prepared/, phase1/outputs/, large phase1/outputs_phase1/*.csv files, or phase2/runs/.

Do not move Phase 2 files back to root-level scripts/, src/, or Data/phase2_params/. The current clean layout is under phase2/.

Current Reference Docs

Use these active docs with this Bible:

- phase1/README.md
- phase1/configs/phase1_config_parameters.md
- phase2/README.md
- phase2/scenarios/README.md
- phase2/docs/PARAMETER_REFERENCE.md
- phase2/docs/SCENARIO_USE_CASES_AND_TESTING.md
- phase2/docs/SCENARIO_SUPPORT_MATRIX.md
- phase2/runs/README.md
- docs/SOG_TECHNICAL_WALKTHROUGH.md
- docs/SOG_V1_TO_V2_COMPARISON.md
- docs/FRONTEND_RUNBOOK.md

Older docs can still be useful for context, but this file is the current single-document Bible for the working tree.